PegaSold: Baku Home Price Prediction

Decision tree, Pegasos SVM and bagging, built from scratch in NumPy

Emiliya Abdulhalimova Aygun Asvarova Murad Bagirzade Kanan Karimli

MLE-AI-201 Machine Learning · AI Academy

October 2026





The problem

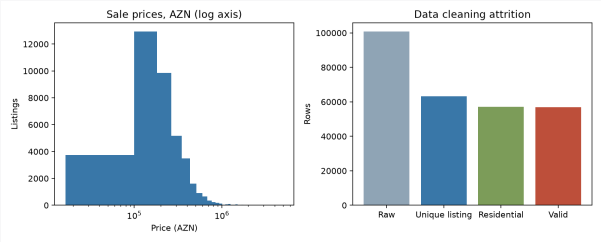
- bina.az listings: 100,775 raw rows, 51 messy Azerbaijani columns
- **Task A:** predict the asking price (we model ln price)
- **Task B:** premium or standard, split at the training median, 209,000 AZN
- Tree, SVM and ensemble are **our own NumPy code**; scikit-learn is only the baseline



Cleaning: 100,775 rows to 57,003

Step	Rows
Raw scrape	100,775
Deduplicate	63,241
Baku, 3 residential types	57,069
Price and area windows	57,003

- Latest scrape per advert; 544 reposts removed
- m^2 for flats, *sot* for land
- Split 70/15/15, seed 42
- 61 automated checks: CLEAN



82 features, no leakage

- 11 continuous: log area, rooms, floors, lat/lng, distance to centre, area per room ...
- 9 binary flags, 3 property types, 59 locations
- Fills, scaling and tier threshold measured on the **training split only**
- Removed `unit_price`, `total_price`, `description`

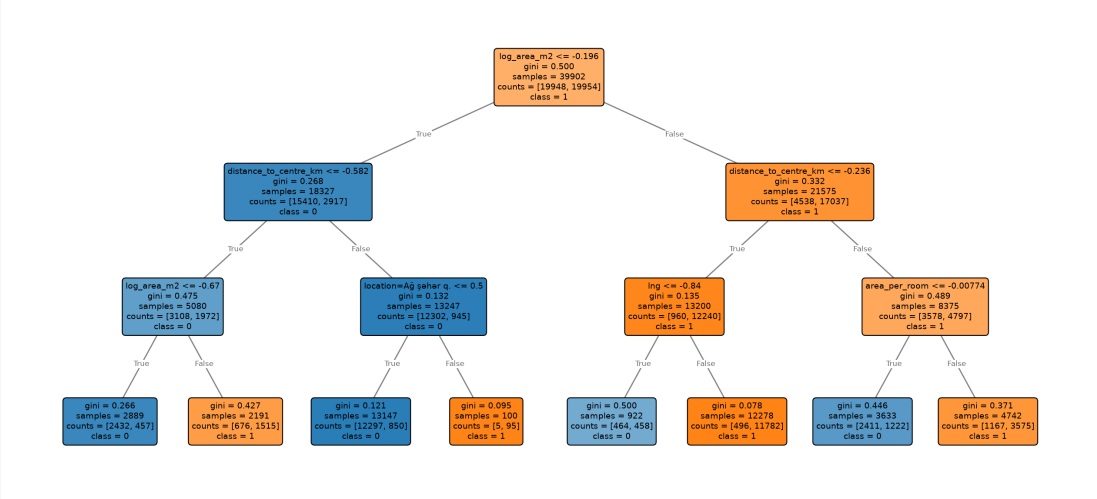


Four models, all from scratch

Decision tree	sorted-prefix split search, Gini / entropy / MSE, depth and leaf limits, class frequencies for `predict_proba`, feature importances
Pegasos SVM	hinge loss + L_2, step $1/(\lambda t)$, projection onto $\|w\| \leq 1/\sqrt{\lambda}$, capped intercept step
RFF SVM	128 random Fourier features approximating an RBF kernel, same Pegasos solver
Bagging	30 trees $\times$ 6,000 bootstrap rows, depth 16, soft voting, out-of-bag score



What a tree learns



Task A: price

Model	RMSE valid	RMSE test	R^2 test
Dummy	0.632	—	—
Ridge	0.241	—	—
scikit-learn tree	0.239	0.244	—
Ours: tree (depth 10)	0.238	0.245	0.856
Ours: bagging	**0.205**	**0.218**	**0.886**

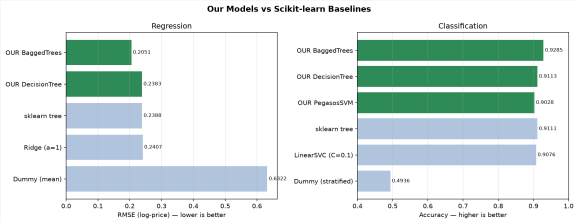
Typical error 17.8% (tree) $\rightarrow$ **14.9%** (bagging). Bagging cuts test MSE by 20.9%.

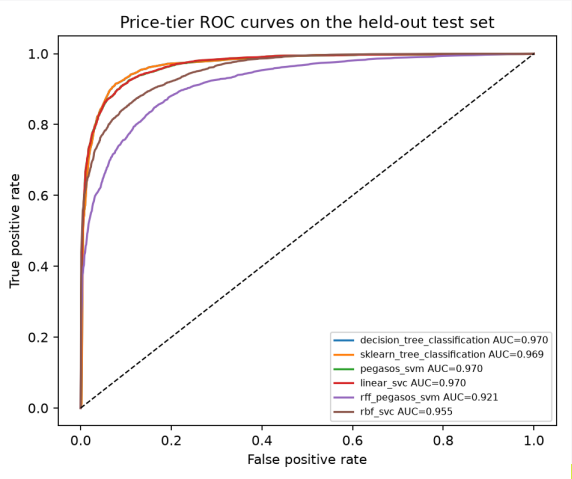


Task B: price tier

Model	Acc test	F1 test	AUC test
scikit-learn tree	0.916	0.917	0.969
Ours: tree (depth 8)	0.916	0.917	0.970
LinearSVC	0.908	0.909	0.970
Ours: Pegasos SVM	0.908	0.909	0.970
RBF SVC (8k rows)	0.875	0.875	0.955
Ours: RFF SVM (8k rows)	0.842	0.844	0.921
Ours: bagging	**0.934**	—	**0.983**

Ours vs scikit-learn



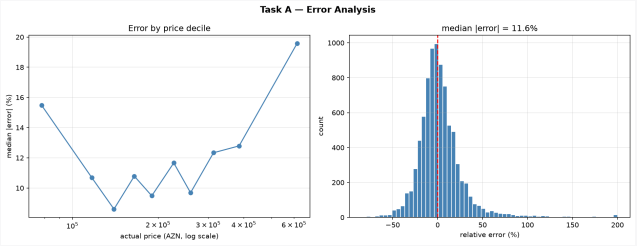


5-fold CV: tree 0.909 ± 0.001 (sklearn 0.909), Pegasos 0.904 ± 0.002 (LinearSVC 0.904).

Two choices we measured

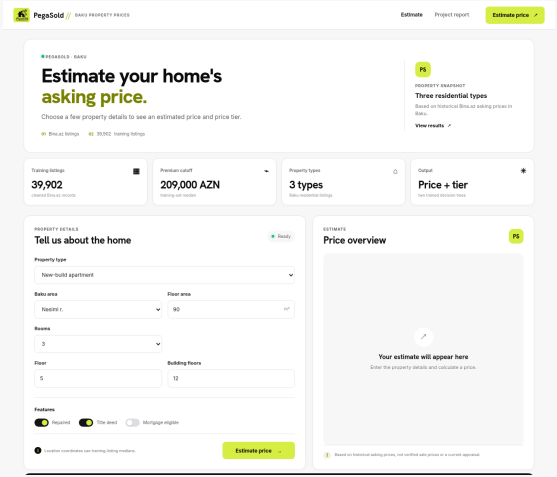
- **Ensemble depth.** Members at depth 8: accuracy 0.914, AUC 0.976. At depth 16: **0.929, 0.981** (+1.4 points). Averaging controls variance, so members should fit hard.
- **SVM regularisation.** λ 0.01 $\rightarrow$ 0.001: accuracy 0.890 $\rightarrow$ **0.903**, AUC 0.957 $\rightarrow$ 0.967.
- **Nonlinear SVM.** RFF did not beat the linear SVM (0.842 vs 0.908): the extra flexibility did not pay off here.

Where the model fails



- Median error 10% for 100k–350k AZN
- 33% above 1M AZN (117 listings)
- Top drivers: log area (0.63), distance to centre (0.24)
- Luxury homes are rare and priced by features we do not see

Live demo



`python -m src.web.web_demo` → price + tier from our trained models

Limits and next steps

- Depth and λ tuned on 6,000 rows, used on 39,902
- Ensemble depth set by one measurement, not its own sweep
- Asking prices from one 2024 scrape, not sale prices
- Next: sweep ensemble size and depth; a separate model for luxury listings



Team

Emiliya Abdulhalimova	data cleaning, EDA, error analysis
Aygun Asvarova	features, scaling, model selection
Kenan Karimli	decision tree, bagging
Murad Bagirzade	SVM, metrics, validation, pipeline

`github.com/b-murad/PegaSold` tag `v1.0-final`

